

AIEElementwiseInc	
	A X Y _descstr context input_bo output_bo size
__init__(A: nengo.builder.Signal, X: nengo.builder.Signal, Y: nengo.builder.Signal, tag) make_step(signals, dt, rng)	

AIELif	
	context dt input_bo output_bo size
__init__(size, dt, tau_rc, tau_ref, min_voltage, amplitude, initial_state) step(dt, J, output, voltage, refractory_time)	

context

context

AIEContext	
	__bos : dict __device __instr_bo __instr_v __kernel __next_gid : int
__init__(device, kernel, instr_v, instr_bo) create_inout_bo(name, volume, itemsize) get_bo(name) get_device() get_kernel() kernel_call()	

AIEConnection	
	_to_neurons function has_weights is_decoded learning_rule post_obj post_slice pre_obj pre_slice size_in size_mid size_out
__init__()	

AIEBuilder	
	builders : dict
__init__() build(model, obj)	

AIEDense	
	_argreprs init_shape size_in size_out
__init__(shape, init)	

AIEManager	
init_aie(xclbin_path): tuple[xrt.device, xrt.kernel] load_insts(insts_path, device, kernel): tuple[list[int], xrt.bo] next_multiple_of(divider)	

AIESimulator	
__init__(network, dt, seed, progress_bar)	

ElementwiseIncBuilder	
	__init__() build(device, size): tuple[str, str]

LIFNeuronBuilder	
	__init__() build(device, size): tuple[str, str]

MlirBuilder	
	__insts_path __mlir_source __xclbin_path mlir_source
__init__() build(device, size): tuple[str, str] compile(): tuple[str, str]	

KernelManager	
	__kernel_objects : dict __kernel_sources : dict resources
compile_all() get_kernel_object_for(opname) register_kernel_source(opname)	

OpNames	
	name